

Security Automation Framework - Full Working (Step by Step)

1. Source Generates Log

- a) User clicks bad link, malware runs, login fails.
- b) Windows / Linux / Firewall writes a raw log.
- c) Example: outbound connection to example.com
- d) This is noise, not yet an alert.

2. Wazuh Agent Forwards It

- a) Agent on laptop watches log files.
- b) Agent ships log to Wazuh Manager on port 1514.
- c) No agent? Firewall sends via syslog.
- d) Manager now has raw event.

3. Wazuh Manager Detects

- a) Manager decodes log to fields: srcip, dst domain, user.
- b) Checks ruleset: if dst bad OR 5 fails in 1 min -> trigger.
- c) Assigns rule.id=100002, level=10, severity=medium.
- d) Writes to /var/ossec/logs/alerts/alerts.json.
- e) For project: you copy that as data/sample_alert.json. That's your input.

4. Python Ingests Alert

- a) main.py runs json.load(open(sample_alert.json)).
- b) Reads alert_id, severity, indicators[].
- c) If file missing / JSON broken -> log error + stop.
- d) Alert is now a dict in memory.

5. Extract + Validate Indicators

- a) Pull indicators = alert.get('indicators', []).
- b) Check present? If empty -> LOW + stop.
- c) Check type in [ip, domain, url, sha256]? If no -> INVALID_INDICATOR.
- d) Check format valid? If no -> stop, don't waste API.
- e) Pass clean example.com / domain forward.

6. Local Rule Check

- a) Read alert.severity.
- b) critical/high -> HIGH.
- c) medium -> MEDIUM.
- d) else -> LOW.

e) Save as local_rule_result. No internet used.

7. VirusTotal Enrichment

- a) Pick URL: domain -> /domains/example.com, ip -> /ip_addresses/, hash -> /files/.
- b) Send GET with header x-apikey: VT_API_KEY.
- c) Get big JSON back.
- d) Parse only last_analysis_stats: malicious, suspicious, harmless.
- e) Normalize to {indicator, malicious:0, suspicious:0, risk:LOW}.
- f) If timeout / 401 / 429 / network fail -> use local result only, don't crash. Cache to avoid repeat.

8. Risk Engine Decides

- a) Take alert + local + vt_result.
- b) if severity critical -> HIGH.
- c) else if malicious >=5 -> HIGH.
- d) else if malicious >0 -> MEDIUM.
- e) else if suspicious >0 -> MEDIUM.
- f) else -> LOW.
- g) Final = LOW for example.com. This is final verdict.

9. SOAR Routes It

- a) Python POSTs final JSON to Shuffle/Tines webhook.
- b) SOAR Node1 Start: receives alert.
- c) Node2 Parse: extracts alert_id, indicator, risk.
- d) Node3 Enrich: already has VT data.
- e) Node4 Decision: HIGH->ticket, MEDIUM->review, LOW->log.
- f) Node5-6: triggers next steps.

10. Slack/Discord Notifies

- a) Build message: ALERT-001 | example.com | LOW | 0 malicious.
- b) requests.post(webhook_url, {text: message}).
- c) If HTTP 200 -> delivered.
- d) HIGH pings loudly + ticket, LOW just logs.

11. SQLite Stores Proof

- a) Open incidents.db.
- b) Insert row: timestamp, alert_id, indicator, type, local, vt_malicious, final_risk, action.
- c) This is audit trail + dashboard source.
- d) Never store real customer data, only synthetic.

12. Dashboard + Demo

- a) Read SQLite -> show table.
- b) Demo flow: Input -> Enrich -> Decide -> Notify -> Evidence.

c) Show examiner audit_log row as proof work reduced from 10 manual clicks to 1 auto-run.